Student Performance & Attendance Analyzer

A PROJECT REPORT

Submitted by

Roll Number	Registration Number	Student Code	Student Name
26102410006	26102000125	BWU/MCA/26/007	Milan Maity

in partial fulfillment for the award of the degree

Of

MASTER OF COMPUTER APPLICATIONS

IN

DEPARTMENT OF COMPUTATIONAL SCIENCES

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125



NOVEMBER 2026

Student Performance & Attendance Analyzer

Submitted by

Roll Number	Registration Number	Student Code	Student Name
26102410006	26102000125	BWU/MCA/26/007	Milan Maity

in partial fulfillment for the award of the degree

of

MASTER OF COMPUTER APPLICATIONS

in

DEPARTMENT OF COMPUTATIONAL SCIENCES



BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

BRAINWARE UNIVERSITY

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

[COMPUTATIONAL SCIENCES]

BONAFIDE CERTIFICATE

Certified that this project report **"Student Performance & Attendance Analyzer"** is the bonafide work **of "Milan Maity"** who carried out the project work under my supervision.

_ _ _ _ _ _ _ _ _ _ _ _	_ _ _ _ _ _ _ _ _ _ _ _
SIGNATURE	**SIGNATURE**
DR. Jayanta Aich	MR. Rahul Kumar Ghosh
HEAD OF THE DEPARTMENT	**SUPERVISOR**
Computational Sciences	Assistant Professor
Brainware University	Computational Sciences

ABSTRACT

This project presents a Python-based application that analyzes student academic performance and attendance data from a CSV file. The system reads student records, calculates key metrics such as average marks, attendance percentage, pass/fail status, and grade distribution, and generates visual charts for better insights. It is developed using modular Python scripts (main.py, analysis.py, charts.py, student.py) and relies on libraries such as pandas, matplotlib, and related data-processing tools.

A simple command-line interface allows users to load data, perform analysis, view statistics, and generate graphical reports without complex coding. The system identifies students with low attendance or poor performance, computes overall class statistics, and produces clear visualizations. This reduces manual calculation effort for teachers and administrators and demonstrates how Python data analysis and visualization techniques can be combined into a practical educational tool.

TABLE OF CONTENTS

Chapter	Title	Page No.
1	Introduction	1
2	Objective	2
3	Planning	3
4	Requirement Analysis	5
5	System Flow	6
6	Proposed Design	13
7	Experimental Result	15
8	Future Scope	18
9	Conclusion	19

LIST OF FIGURES

Figure No.	Title	Page No.
5.1	Context Level Diagram	6
5.2	Data Flow Diagram Level 0	7
5.3	Data Flow Diagram Level 1	8
5.4	Use Case Diagram	9
5.5	Activity Diagram	10
5.6	Sequence Diagram	11
5.7	Flow Chart	12
7.1	Data Loading	15
7.2	Student Modeling	15
7.3	Analysis	16
7.4	Bar Chart Generation	16
7.5	Line Graph Generation	17
7.6	Histogram Generation	17

LIST OF SYMBOLS, ABBREVIATIONS, AND NOMENCLATURE

Symbol / Abbreviation	Description
CSV	Comma Separated Values (file format used for student data input)
DFD	Data Flow Diagram
CLD	Context Level Diagram
UML	Unified Modeling Language
GUI	Graphical User Interface
PDF	Portable Document Format
%	Percentage (used for Attendance Percentage)
Avg	Average (used for Average Marks)
ID	Identifier (Student ID)
→	Data flow / Message direction (in DFD and Sequence Diagram)
○ / ▭	Process / Activity symbol (in Flow Chart and Activity Diagram)
◇	Decision symbol (in Flow Chart and Activity Diagram)
▭ (open)	Data Store (in DFD)
Actor	External entity / User (in Use Case and Context diagrams)

CHAPTER 1: INTRODUCTION

Traditional student performance and attendance tracking relies on manual registers, spreadsheets, and time-consuming calculations. Teachers often struggle to quickly identify students with low attendance or poor academic results, and generating summary reports or visual insights requires additional effort. With the availability of Python data analysis libraries, it is now possible to automate large parts of this process.

This project focuses on building a system that reads student data (marks and attendance) from a CSV file, processes the records, calculates important metrics, and produces both textual statistics and graphical charts. The idea is simple: the user provides a structured data file, and the program computes averages, attendance percentages, grades, pass/fail status, and class-level summaries, then visualizes the results.

The system uses a modular design with separate files for data modeling, analysis logic, chart generation, and the main program entry point. Behind the interface, the workflow follows clear steps: load data, validate records, compute performance and attendance metrics, generate visualizations, and present a consolidated report. The project demonstrates how data processing, statistical calculation, and visualization can be combined into a lightweight, easy-to-understand educational analytics tool suitable for students and beginners.

CHAPTER 2: OBJECTIVE

The objective of this project is to develop a Python-based Student Performance & Attendance Analyzer that enables users to process student records from a CSV file and obtain meaningful academic and attendance insights. The system aims to provide a modular, automated workflow by integrating data loading, metric calculation, and visualization components. Users can load student data, compute individual and class-level statistics (average marks, attendance percentage, grades, pass/fail status), identify under-performing or low-attendance students, and generate charts without requiring advanced programming knowledge. By combining pandas for data handling, custom analysis functions, and matplotlib-based visualizations, the project intends to simplify student monitoring, reduce manual calculation effort, and offer an accessible end-to-end performance and attendance analysis pipeline.

CHAPTER 3: PLANNING

The planning phase outlines the step-by-step approach used to design, organize, and implement the Student Performance & Attendance Analyzer. Proper planning ensures that the workflow remains efficient, modular, and easy for beginners to understand and use.

1. Requirement Identification

- Understand the need for a simple Python tool to analyze student marks and attendance from CSV data.
- Identify the target users (teachers, students, academic administrators, beginners).
- Define the core requirement: CSV data → analysis of performance & attendance → statistics + charts.

2. System Design Planning

- Decide to use pandas for data handling and matplotlib for visualization.
- Choose a modular structure with separate files for Student class, analysis logic, chart generation, and main program.
- Plan the workflow from data loading to final report and charts.
- Select supporting libraries listed in requirements.txt.

3. Pipeline Structure

Design a sequential pipeline with the following steps:

- Data loading from students.csv
- Student object creation / data validation
- Performance metric calculation
- Attendance percentage calculation
- Grade assignment and pass/fail determination
- Statistical summaries
- Chart generation
- Result display

4. File & Module Planning

- student.py – Student data model / class
- analysis.py – Core calculation and analysis functions
- charts.py – Visualization functions
- main.py – Entry point and user interaction / menu
- students.csv – Sample input data
- requirements.txt – Dependencies

5.Code Structure Planning

- Break the code into clear, reusable sections:
- Data loading and validation
- Metric calculation functions
- Filtering of low-attendance / low-performance students
- Chart plotting functions
- Main menu / command-line interface

6.Testing & Validation Plan

- Test with sample CSV data.
- Verify correct average, percentage, and grade calculations.
- Check chart generation for different metrics.
- Validate handling of edge cases (missing data, zero attendance, etc.).

7.Scalability & Improvement Planning

Plan for expandable features such as:

- Support for multiple subjects or semester data
- Export of reports to PDF or Excel
- Interactive GUI (Tkinter or Streamlit)
- Predictive analysis using simple machine learning
- Database integration instead of CSV

CHAPTER 4: REQUIREMENT ANALYSIS

❑ Functional Requirements

1.Data Input

- The system must read student records from a CSV file (students.csv).
- Each record typically includes student identifiers, marks, and attendance information.
- Input is handled through Python file reading and pandas DataFrame operations.

2.User Interaction

- A simple command-line interface or menu-driven program in main.py.
- Options to load data, run analysis, view statistics, generate charts, and exit.
- Users should not need to write code to obtain results.

3.Model and Framework Requirements

The project depends on the following Python libraries (as listed in requirements.txt):
- pandas (data loading and manipulation)
- matplotlib (chart generation)
- Other standard libraries (os, csv, etc.) as needed
- Custom modules: student.py, analysis.py, charts.py

4.Analysis Requirements

For every student record:
- Calculate average marks / total performance score.
- Calculate attendance percentage.
- Assign grades based on marks.
- Determine pass/fail status (considering both marks and minimum attendance threshold, typically 75%).
- Identify students with low attendance or poor performance.

5.Visualization Requirement

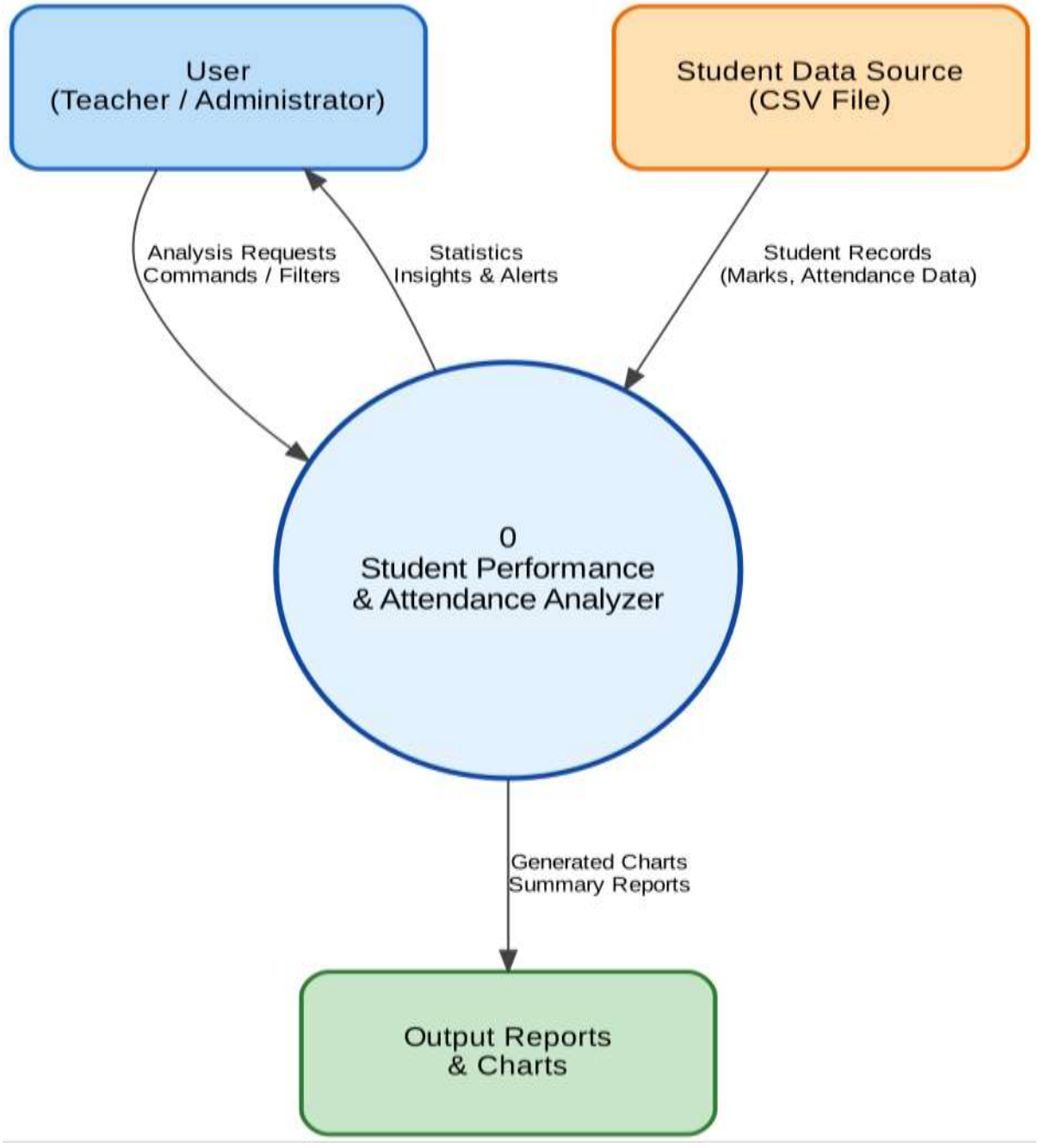
- Generate bar charts, pie charts, or other suitable plots for grade distribution, attendance ranges, performance comparison, etc.
- Charts should be saved or displayed clearly.

6.Output Organization

The system must:
- Display textual summaries and statistics on the console.
- Generate and save chart images.
- Provide a clear final report of class-level and individual insights.

CHAPTER 5: SYSTEM FLOW

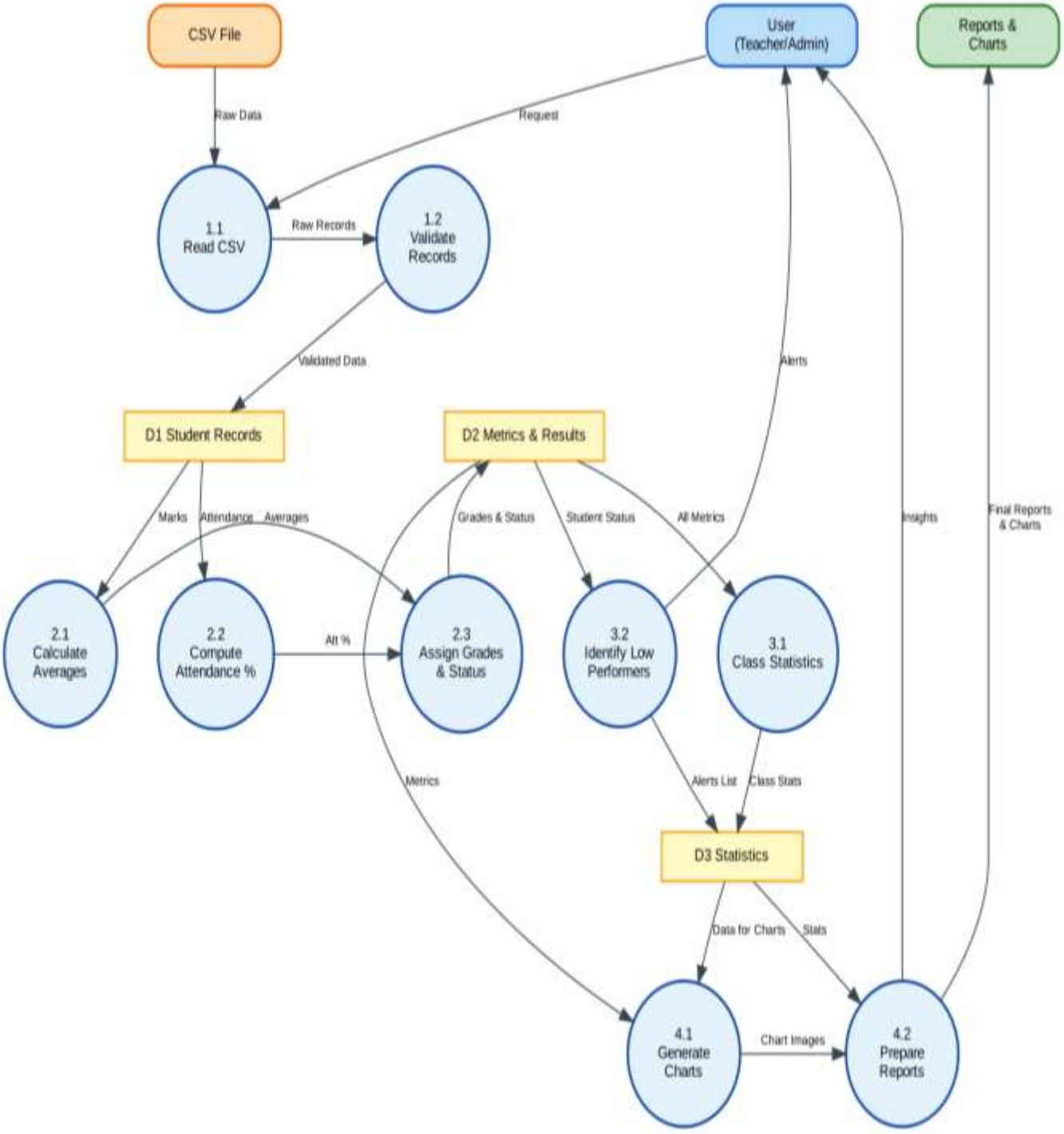
Context Level Diagram (CLD):



It shows the overall system as a single process interacting with external entities (User and CSV data source) and producing reports/charts.

Fig 5.1: Context Level Diagram

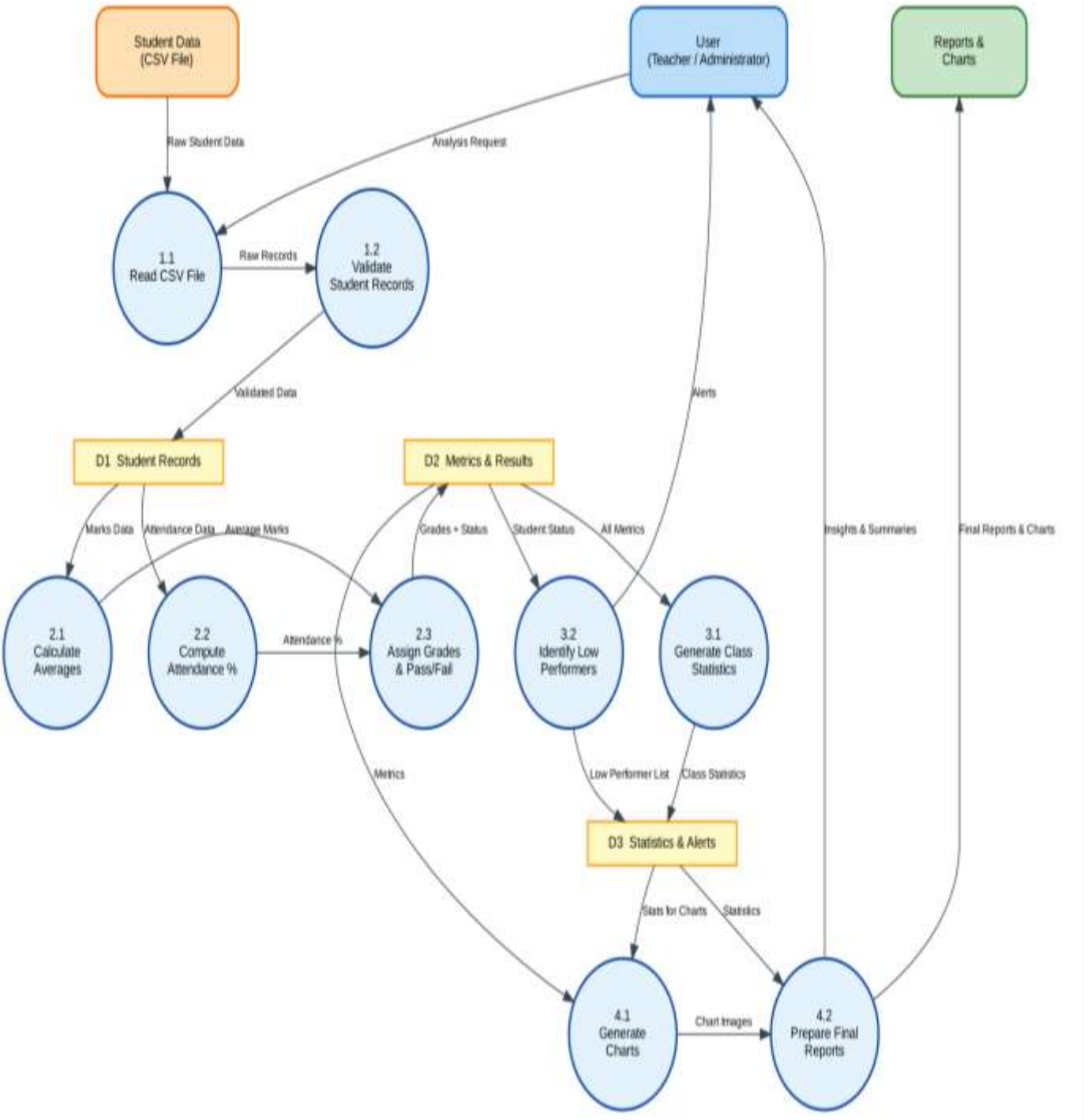
Data Flow Diagrams (DFD):



It illustrates the major processes (Load Data, Analyze, Generate Statistics, Create Charts) along with data stores and high-level data flow.

Fig 5.2: Data Flow Diagram Level 0

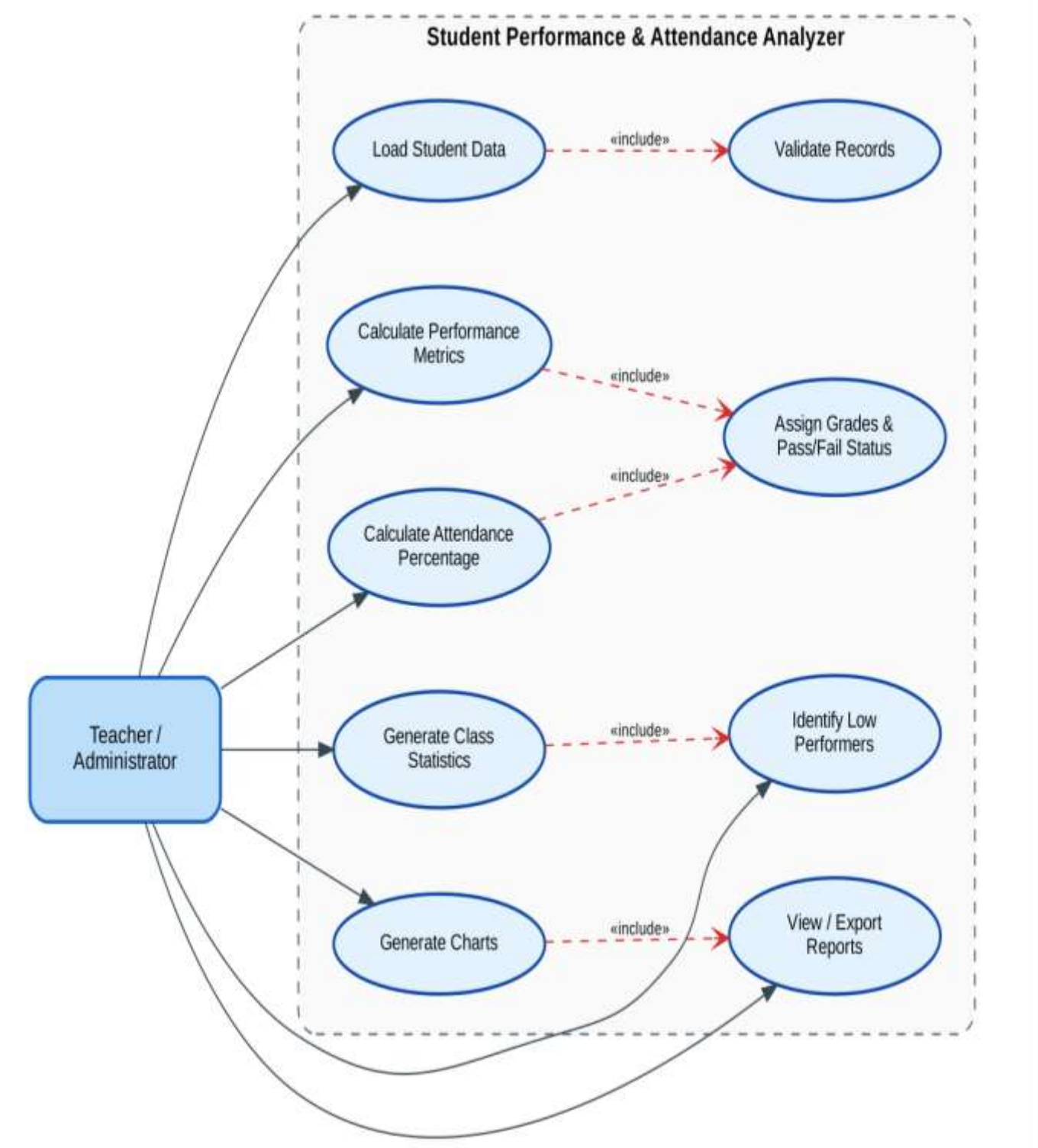
Data Flow Diagrams (DFD):



It breaks down the Level 0 processes into detailed sub-processes for data loading, metric calculation, statistics generation, and chart creation.

Fig 5.3: Data Flow Diagram Level 1

Use Case Diagram:



It depicts the interactions between the Teacher/Administrator and the system's key functionalities such as loading data, analyzing performance, and generating reports.

Fig 5.4: Use Case Diagram

Activity Diagram:

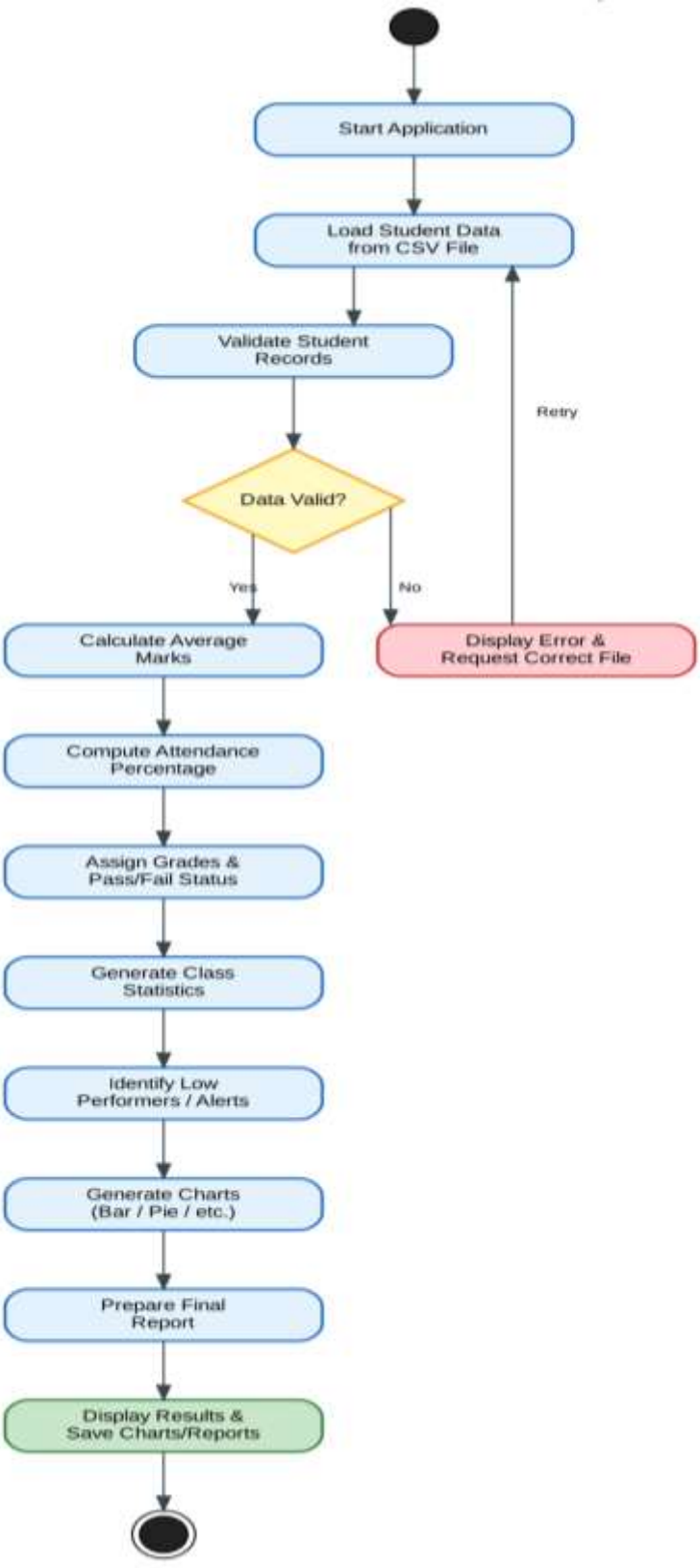


Fig 5.5: Activity Diagram

It represents the step-by-step workflow of the system from starting the application to displaying the final results, including decision points for data validation.

Sequence Diagram:

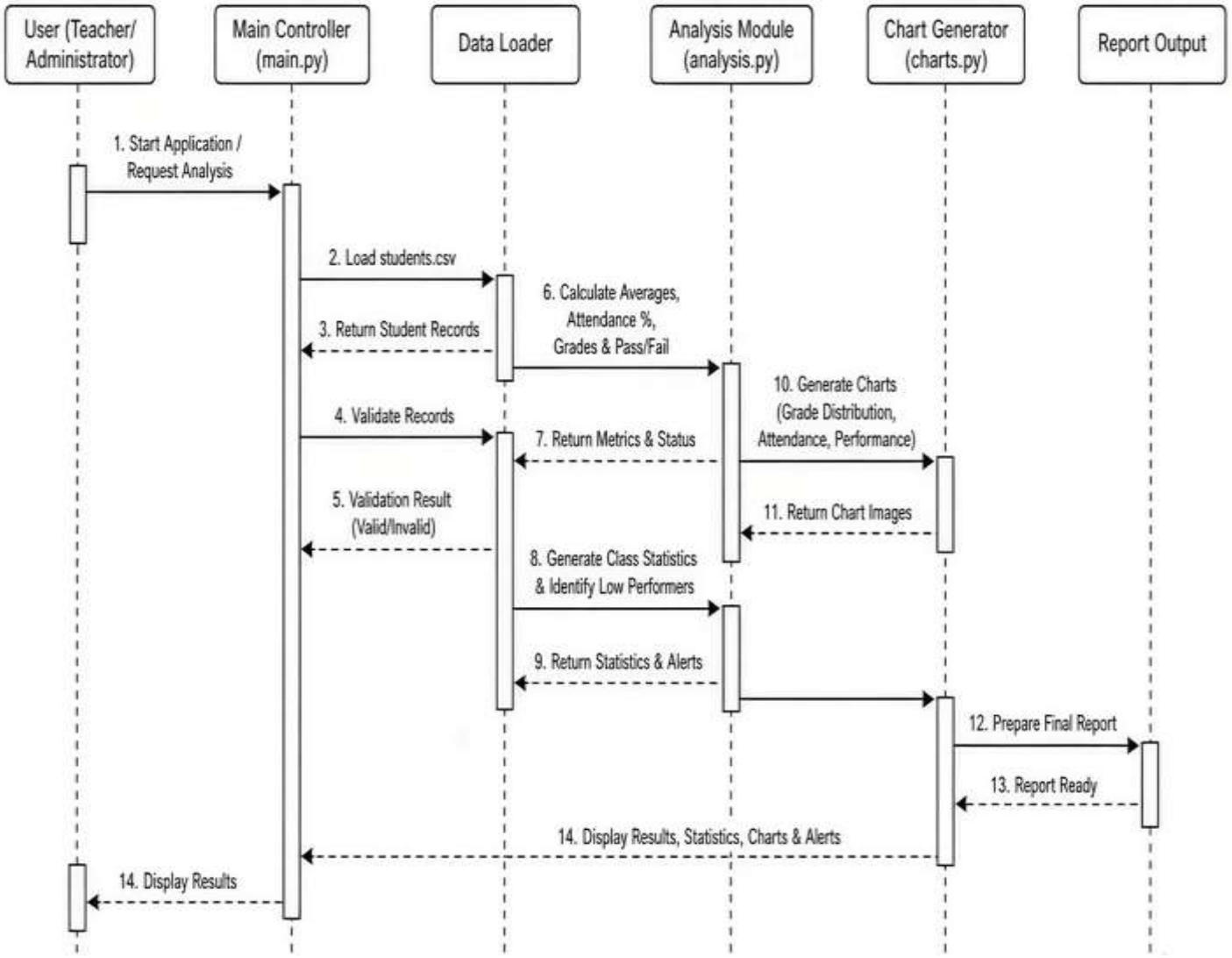


Fig 5.6: Sequence Diagram

It shows the chronological sequence of messages exchanged between the User, Main Controller, Data Loader, Analysis Module, Chart Generator, and Report Output.

Flow Chart:

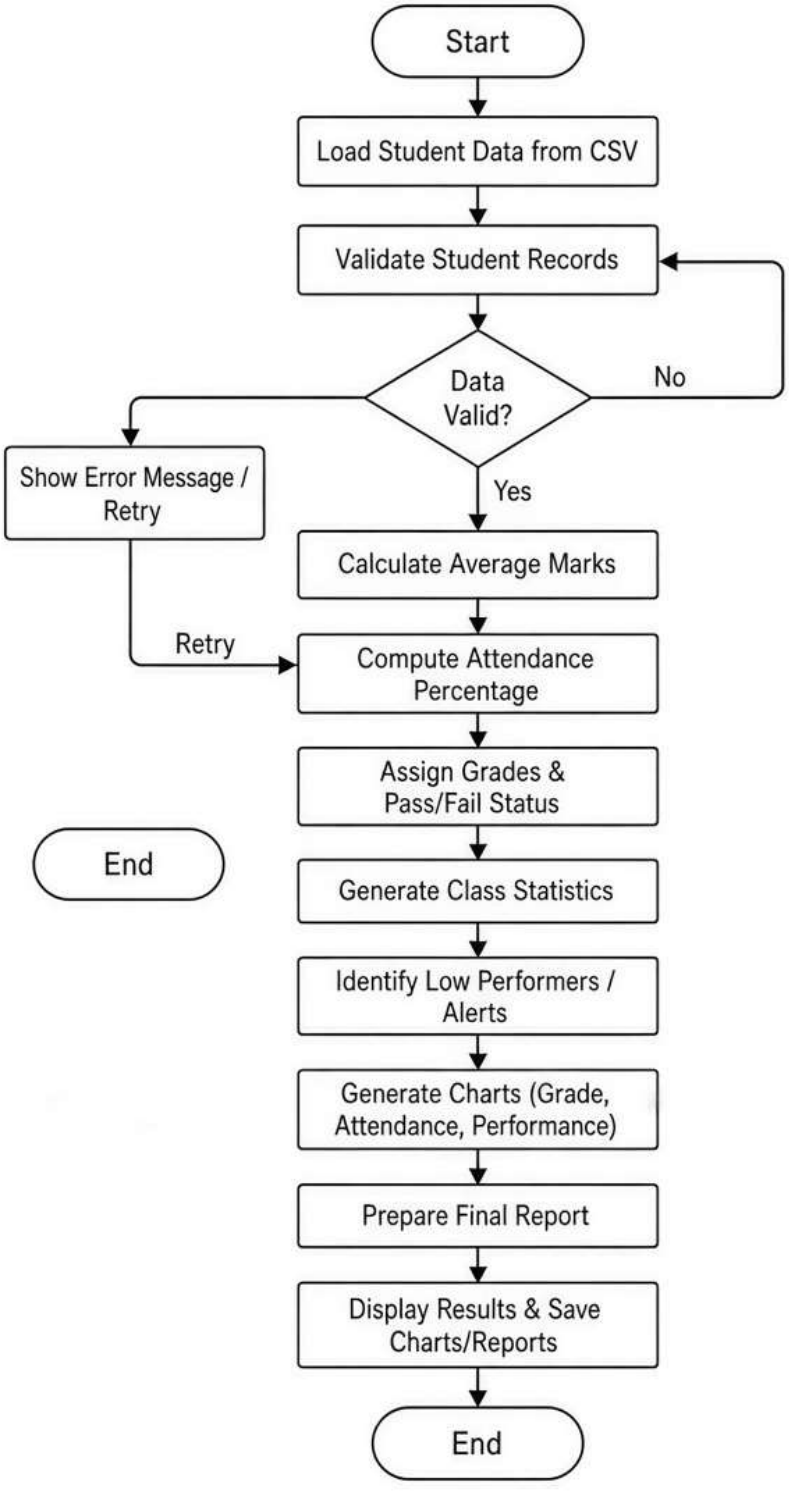


Fig 5.7: Flow Chart

It presents the complete sequential logic of the system using standard flowchart symbols, including decision points for data validation and the path to final report generation.

CHAPTER 6: PROPOSED DESIGN

This chapter explains how the system processes student data and produces analysis results and charts. The method follows a sequential pipeline that begins with CSV input and ends with statistics and visualizations. The overall workflow is shown in the diagrams, and each stage is described in detail.
The application loads student records from students.csv, creates structured data objects, computes performance and attendance metrics, filters important cases, and generates charts. This method allows users to obtain insights from academic data without manual calculations or complex tools.

The process includes the following key stages:
1. Data loading from CSV
2. Student data modeling
3. Performance and attendance analysis
4. Statistical summary generation
5. Chart creation and display
6. Final report presentation

The steps included are –

Step 1: Data Loading

The program reads the students.csv file using pandas or standard CSV handling and prepares a clean dataset for analysis.

Step 2: Student Modeling

A Student class (student.py) represents individual records with attributes such as name/ID, marks, and attendance. Objects or DataFrame rows are created for further processing.

Step 3: Analysis

The analysis module (analysis.py) performs:
- Calculation of average marks
- Attendance percentage computation
- Grade assignment
- Pass/fail determination (based on marks threshold and minimum attendance, commonly 75%)
- Identification of students needing attention
- Class-level aggregates (overall average, highest/lowest performers, attendance statistics)

Step 4: Chart Generation

The charts module (charts.py) uses matplotlib to produce visual representations such as:
- Grade distribution pie/bar charts
- Attendance percentage histograms or bar charts
- Performance comparison plots
- Scatter or correlation views (attendance vs marks) when applicable

Step 5: Main Program Flow

main.py orchestrates the process: loads data, calls analysis functions, triggers chart generation, and presents results through a simple menu or sequential execution.

Step 6: Final Output

Users receive console statistics, lists of low-attendance or low-performance students, and generated chart images. This supports both verification and decision-making.

This methodology is effective because:
- Data processing is automated and repeatable
- Modular design makes the code easy to understand and extend
- Visualizations improve insight communication
- No external specialized software is required beyond standard Python libraries
- The workflow is suitable for undergraduate-level implementation while demonstrating practical data analysis techniques

REPRESENTATIVE CODE STRUCTURE (based on project files)

- **student.py**: Defines the Student class or data structure.
- **analysis.py**: Contains functions for calculating averages, percentages, grades, pass/fail status, and summaries.
- **charts.py**: Contains functions that generate and save/display matplotlib charts.
- **main.py**: Entry point that loads data, invokes analysis and chart modules, and handles user interaction.
- **students.csv**: Sample dataset used for testing and demonstration.
- **requirements.txt**: Lists required packages (pandas, matplotlib, etc.).

CHAPTER 7: EXPERIMENTAL RESULT

The results focus on how the CSV data was processed, how performance and attendance metrics were calculated, how statistics were produced, and how charts were generated. Screenshots of console outputs, charts, and sample reports can be inserted in the respective places.

1. Data Loading Stage

The system successfully reads the students.csv file and prepares the data for analysis.

```
========================================
   STUDENT PERFORMANCE & ATTENDANCE
              ANALYZER
========================================
1. Add Student
2. Show Students
3. Low Attendance Students
4. Sort Students by Marks
5. Save Data
6. Read CSV Data
7. Statistics
8. Graphical Reports
9. Exit

Enter your choice: 6

==================================
        STUDENT CSV RECORDS
==================================
 Roll No        Name  Python  DBMS  Math   AI  Total  Average  Percentage Grade  Attendance
       7 arnab maity    48.0  57.0  64.0 94.0  263.0    65.75       65.75     C       64.29
```

Fig 7.1: Data Loading Stage

2. Performance Analysis

Average marks, grades, and pass/fail status are calculated for each student and for the class overall.

```
Enter your choice: 1

===== ADD STUDENT =====
Enter Roll No: 007
Enter Student Name: Milan Maity

Enter marks out of 100:
Python: 85
DBMS: 75
Math: 95
AI: 86

Enter Total Classes: 50
Enter Attended Classes: 40

Student added successfully!

----------------------------
Roll No     : 007
Name        : Milan Maity
Python      : 85.0
DBMS        : 75.0
Math        : 95.0
AI          : 86.0
Total       : 341.0
Average     : 85.25
Percentage  : 85.25 %
Grade       : A
Attendance  : 80.0 %
----------------------------
```

Fig 7.2: Performance Analysis

3. Attendance Analysis

Attendance percentages are computed, and students falling below the threshold are identified.

.

```
========================================
   STUDENT PERFORMANCE & ATTENDANCE
              ANALYZER
========================================
1. Add Student
2. Show Students
3. Low Attendance Students
4. Sort Students by Marks
5. Save Data
6. Read CSV Data
7. Statistics
8. Graphical Reports
9. Exit

Enter your choice: 3
Enter attendance threshold (%): 45

===== STUDENTS BELOW 45.0% ATTENDANCE =====
No student found below the threshold.
```

Fig 7.3: Attendance Analysis

4. Charts Generation

Visual charts (grade distribution, attendance ranges, performance comparison) are produced.

.

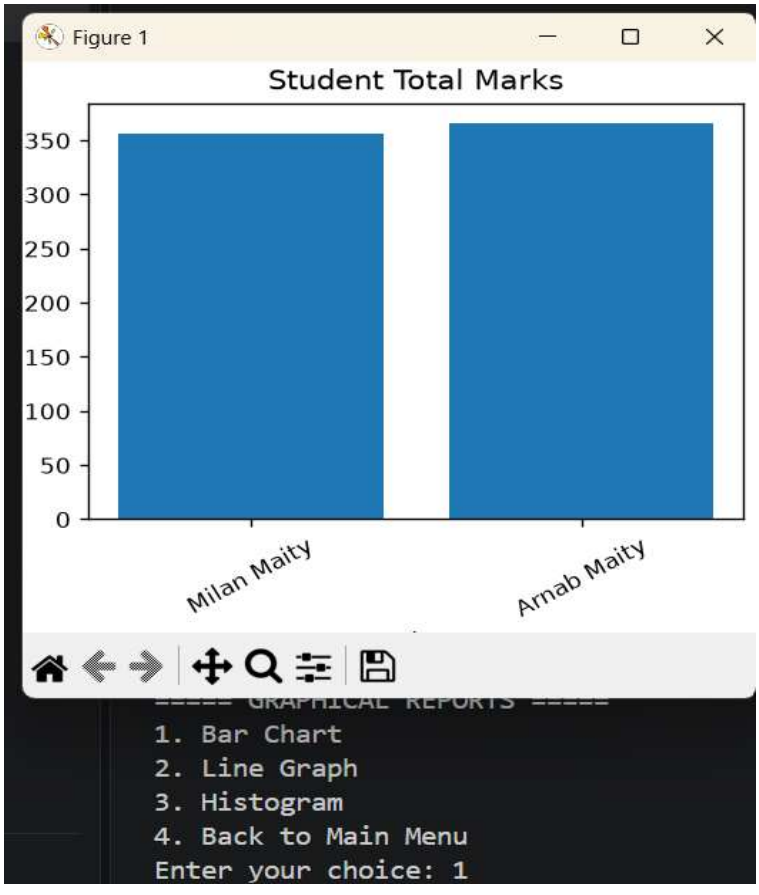


Fig 7.4: Bar Chart Generation

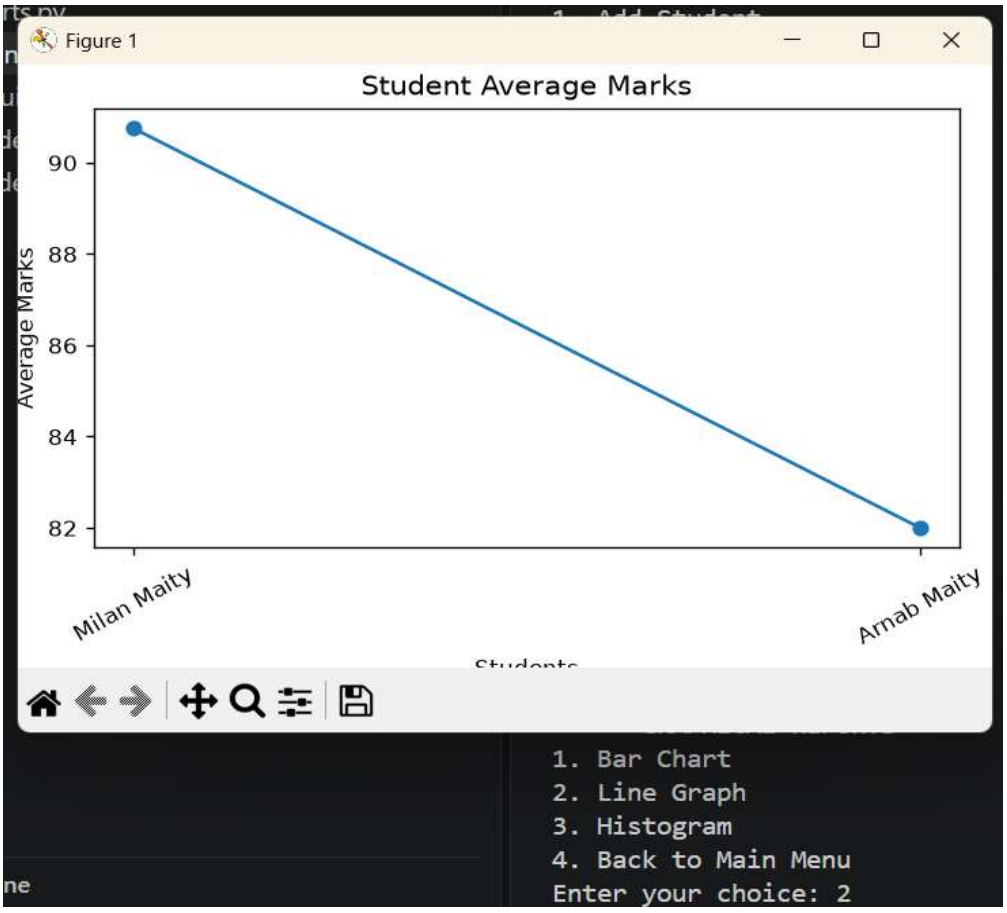


Fig 7.5: Line Graph Generation

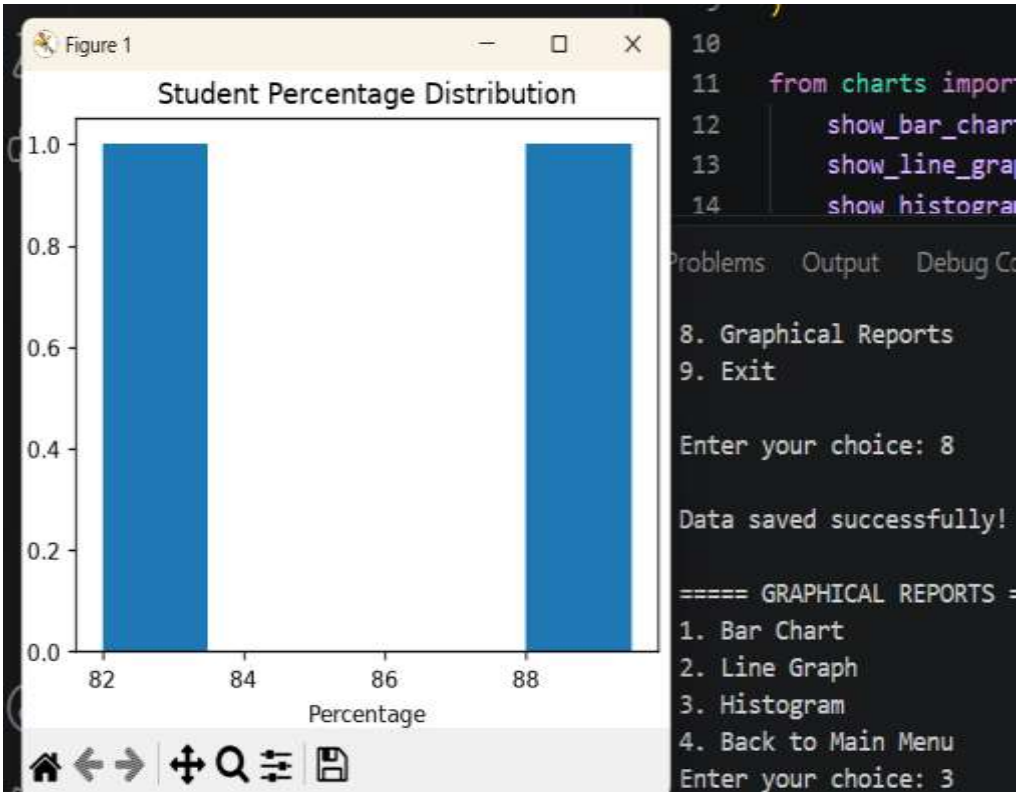


Fig 7.6: Histogram Generation

CHAPTER 8: FUTURE SCOPE

Several improvements can be added in later versions of the project:

1. **Enhanced User Interface**
   Develop a graphical user interface (Tkinter, Streamlit, or web-based dashboard) for easier data upload, interactive filtering, and real-time visualization.

2. **Multi-Subject & Multi-Semester Support**
   Extend the data model to handle multiple subjects, continuous assessment components, and semester-wise tracking.

3. **Report Export Features**
   Add options to export analysis results and charts to PDF, Excel, or PowerPoint for easy sharing with administrators and parents.

4. **Predictive Analytics**
   Incorporate simple machine learning models to predict future performance or identify at-risk students based on historical patterns.

5. **Database Integration**
   Replace or supplement the CSV input with a proper database (SQLite/MySQL) for larger datasets and multi-user access.

6. **Notification & Alert System**
   Add automatic alerts or email notifications for students with critically low attendance or declining performance.

7. **Advanced Visualizations**
   Include more sophisticated charts, heatmaps, trend lines over time, and interactive dashboards.

CHAPTER 9: CONCLUSION

The project successfully showcases a complete, end-to-end workflow for analyzing student academic performance and attendance from structured data through an automated and modular Python pipeline. Beginning with CSV input, the system efficiently loads records, computes meaningful metrics (averages, percentages, grades, pass/fail status), identifies students requiring attention, and generates clear visualizations. This demonstrates that practical educational analytics can be achieved using standard Python libraries when the process is divided into clear, modular stages.

Moreover, the modular design (separate files for data modeling, analysis, charts, and main logic) significantly enhances maintainability and usability, allowing individuals with basic Python knowledge to understand and extend the system. The design emphasizes accessibility and practicality, ensuring that performance and attendance monitoring is not limited to complex enterprise tools. Overall, the project meets its primary objective by delivering a functional, data-driven Student Performance & Attendance Analyzer that is both practical and educationally valuable.

REFERENCES

1. pandas Development Team. pandas: Powerful data structures for data analysis. Available at: https://pandas.pydata.org/

2. Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. Computing in Science & Engineering. Available at: https://matplotlib.org/

3. Python Software Foundation. Python Language Reference. Available at: https://www.python.org/

4. Project source code and files. Available at: https://1drv.ms/f/c/E3468DA91C827B57/IgDEq5627sfITL5RbggJ8layAY2qnA83_JSPa1BnVoBkaMs